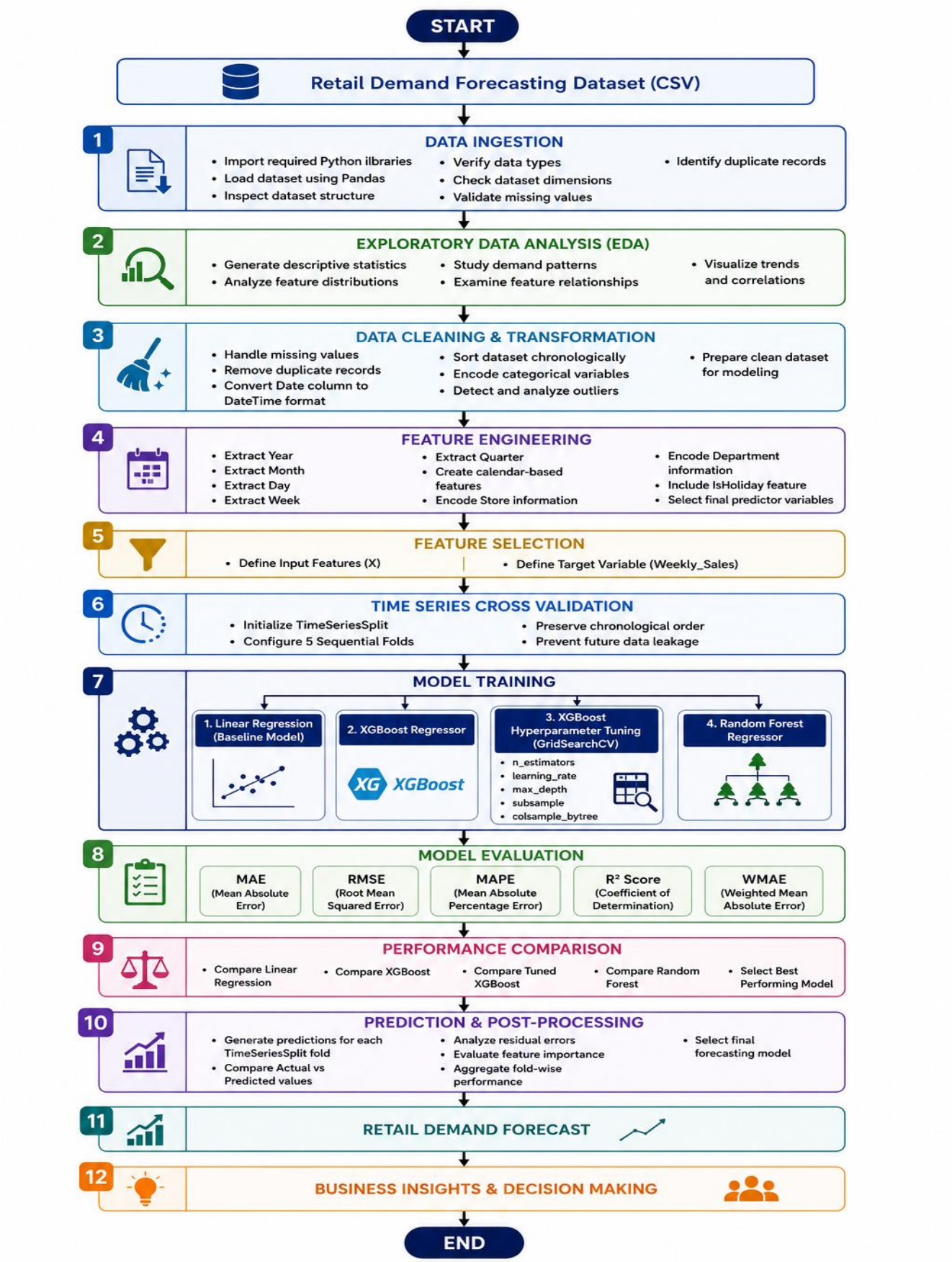
Process Flow Diagram:



Explanation of the Process Flow Diagram

Start

The process begins with the retail demand forecasting problem. The objective is to predict future **Weekly Sales** accurately using historical retail data and machine learning models.

1. Retail Demand Forecasting Dataset (CSV)

The project starts with the retail dataset stored in CSV format. The dataset contains historical sales information along with several explanatory variables such as:

- Store
- Department
- Date
- Weekly Sales
- Holiday Indicator (IsHoliday)
- Temperature
- Fuel Price
- CPI
- Unemployment

These variables provide the information required to understand demand patterns and train forecasting models.

Output: Raw retail dataset

2. Data Ingestion

The first technical stage involves importing and validating the dataset.

Activities Performed:

- Imported Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, and XGBoost.
- Loaded the CSV dataset into a Pandas DataFrame.
- Verified the dataset dimensions.
- Examined column names and data types.
- Identified missing values.
- Checked for duplicate records.

This step ensures that the dataset has been loaded correctly before any preprocessing begins.

Output: Validated dataset ready for analysis.

3. Exploratory Data Analysis (EDA)

EDA helps understand the characteristics of the dataset before model development.

Activities Performed:

- Generated descriptive statistics.
- Examined numerical feature distributions.
- Visualized sales trends.
- Studied relationships between variables.
- Identified patterns and seasonal behavior.
- Used graphs and correlation analysis to understand feature interactions.

EDA provides insights into how different factors influence weekly sales.

Output: Better understanding of the data and business problem.

4. Data Cleaning & Transformation

This stage improves the quality of the dataset before model training.

Activities Performed:

- Handled missing values appropriately.
- Removed duplicate records.
- Converted the Date column into DateTime format.
- Sorted the dataset chronologically.
- Encoded categorical variables into numerical values.
- Examined outliers and retained valid seasonal spikes.
- Prepared the final clean dataset for modeling.

This ensures that the machine learning algorithms receive clean and consistent data.

Output: Cleaned and transformed dataset.

5. Feature Engineering

Feature engineering creates meaningful predictors that improve forecasting performance.

Activities Performed:

From the Date column, several new features were extracted:

- Year
- Month
- Week
- Quarter

Additional features included:

- Store
- Department
- IsHoliday

These engineered features help the model learn seasonal trends and customer purchasing behavior.

Output: Enhanced feature set for model training.

6. Feature Selection

At this stage, the dataset is divided into input variables and the prediction target.

Input Features (X)

The predictor variables include:

- Store
- Department
- Temperature
- Fuel Price
- CPI
- Unemployment
- IsHoliday
- Year
- Month
- Week
- Quarter

Target Variable (y)

- Weekly Sales

This separation prepares the data for machine learning.

Output: Feature matrix (X) and target variable (Weekly Sales).

7. Time Series Cross Validation

Unlike random train-test splitting, demand forecasting requires preserving the chronological order of observations.

Therefore, **TimeSeriesSplit** was used.

Activities Performed:

- Initialized **TimeSeriesSplit**.

- Configured **5 sequential folds**.
- Trained the model on earlier time periods.
- Validated the model on later time periods.
- Prevented future information from leaking into training.

This approach better simulates real-world forecasting where future sales are predicted using only past data.

Output: Sequential training and validation datasets.

8. Model Training

Multiple machine learning models were trained and compared.

Model 1: Linear Regression

Linear Regression was implemented as the baseline model. It establishes a reference point for comparing more advanced algorithms.

Model 2: XGBoost

XGBoost is a powerful gradient boosting algorithm capable of modeling complex nonlinear relationships in retail sales data.

Hyperparameter Tuning (GridSearchCV)

To improve XGBoost performance, GridSearchCV was applied.

The following parameters were optimized:

- n_estimators
- learning_rate
- max_depth
- subsample
- colsample_bytree

TimeSeriesSplit was used during tuning to ensure proper validation.

Model 3: Random Forest

Random Forest was trained as another ensemble model. It combines multiple decision trees and averages their predictions, improving robustness and reducing overfitting.

Output: Three trained forecasting models.

9. Model Evaluation

The trained models were evaluated using several regression metrics.

Mean Absolute Error (MAE)

Measures the average absolute difference between actual and predicted sales.

Lower MAE indicates better prediction accuracy.

Root Mean Squared Error (RMSE)

Measures prediction error while giving greater weight to larger errors.

Lower RMSE indicates a more accurate model.

Mean Absolute Percentage Error (MAPE)

Measures prediction error as a percentage of actual sales.

Lower MAPE indicates more reliable forecasting.

R^2 Score

Measures how well the model explains variation in weekly sales.

Higher values (closer to 1) indicate better performance.

Weighted Mean Absolute Error (WMAE)

Provides weighted error measurement, which is particularly useful when certain observations (such as holiday sales) are more important than others.

Output: Performance metrics for each model.

10. Performance Comparison

The performance of all developed models was compared using the evaluation metrics.

The following models were compared:

- Linear Regression
- XGBoost with HyperParameter Tuning

- Random Forest

The model with the best balance of accuracy and generalization was selected as the final forecasting model.

Output: Best-performing model selected.

11. Prediction & Post-Processing

The selected model was used to generate predictions.

Activities Performed:

- Generated predictions for each TimeSeriesSplit validation fold.
- Compared actual and predicted weekly sales.
- Performed residual error analysis.
- Analyzed feature importance.
- Aggregated fold-wise evaluation results.
- Finalized the forecasting model.

This stage confirms that the model performs consistently across different time periods.

Output: Final demand predictions.

12. Retail Demand Forecast

The selected machine learning model generates forecasts for future weekly sales.

These forecasts help estimate product demand before future sales occur.

Output: Forecasted Weekly Sales.

13. Business Insights & Decision Making

The predicted demand can support several business decisions, including:

- Inventory management
- Stock replenishment
- Promotion planning
- Holiday demand preparation
- Workforce scheduling
- Supply chain optimization
- Revenue forecasting

Accurate demand forecasting helps retailers reduce stock shortages, minimize excess inventory, and improve overall business efficiency.